CYBERSECURITY INTERNSHIP

Name: **Hammad Rasheed**

ID: **DHC-5590**

Week 1

Security Assessment

Step 1

Setup the Application

Prerequisites

- Node.js installed
- npm installed

Installation & Startup Commands

```
# Clone the repository
git clone https://github.com/juice-shop/juice-shop.git --
depth 1

# Enter the directory
cd juice-shop

# Install dependencies (first time only)
npm install

# Start the application
npm start
```

Open the application in your browser:

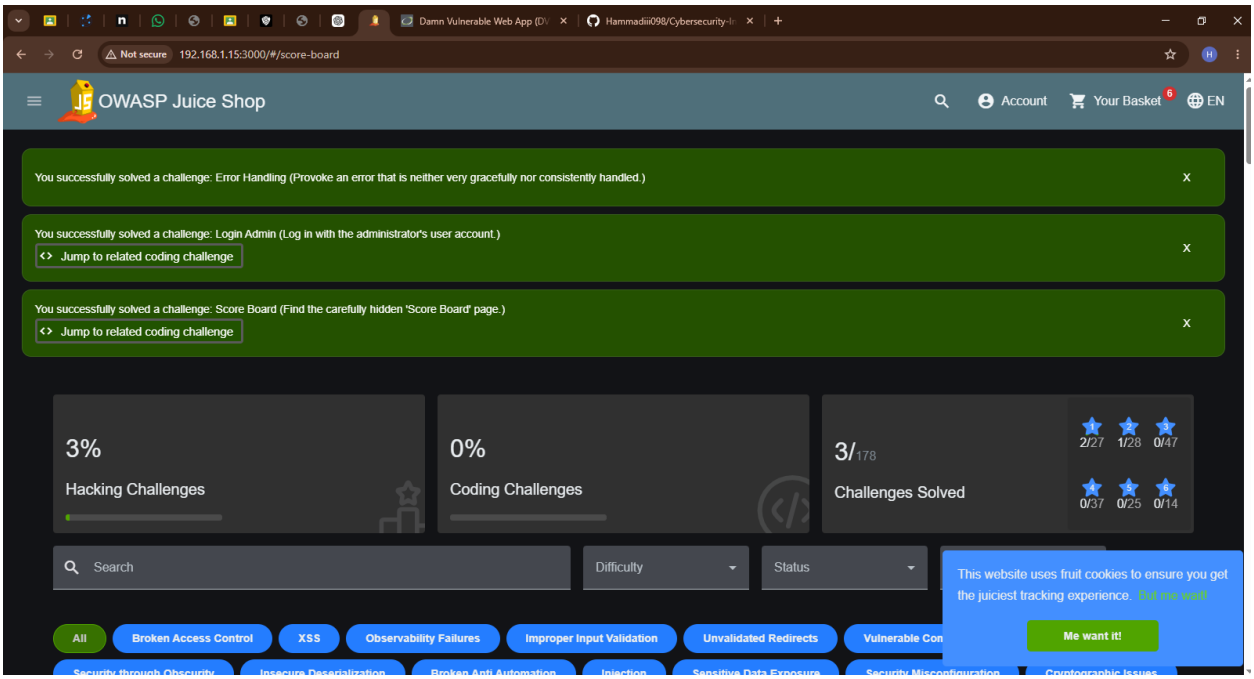
http://localhost:3000

Explore the following sections:

- Signup
- Login
- Search
- Basket
- Profile

Score Board URL:

http://localhost:3000/#/score-board



Step 2: Vulnerability Assessment

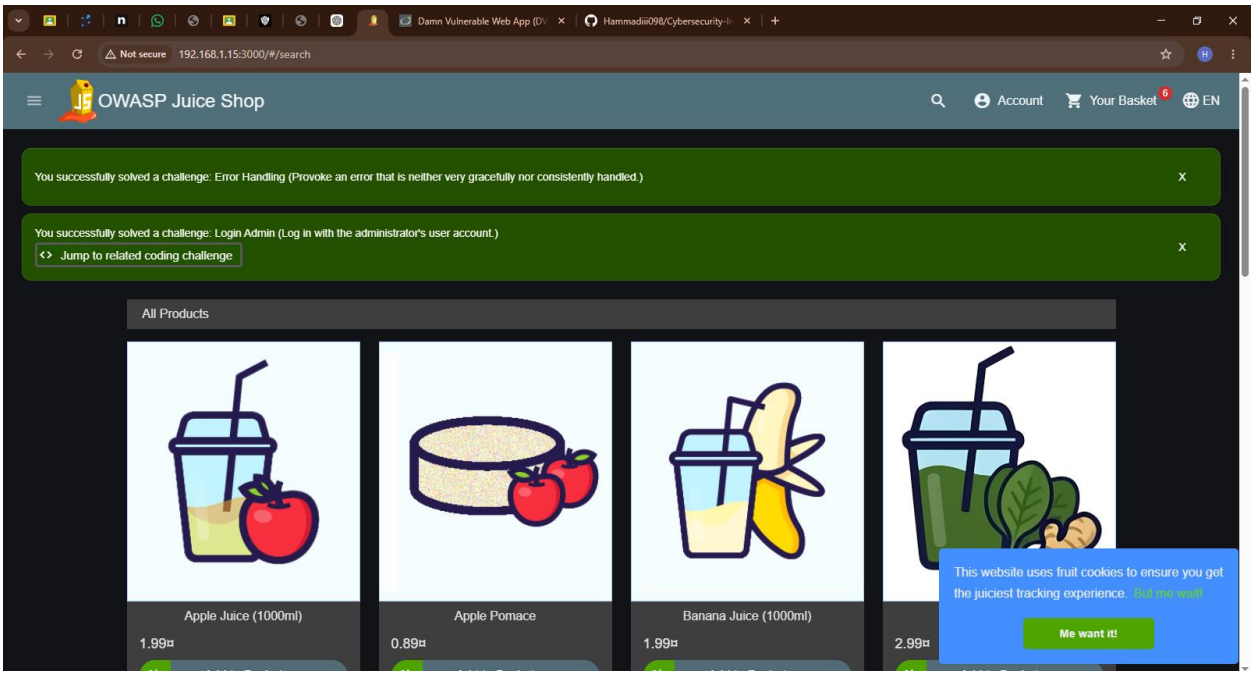
A. SQL Injection — Login as Admin

Steps

1. Open the Login page
2. Enter the following payload in the Email field:

```
' OR 1=1;--
```

3. Enter any password
4. Click **Log in**



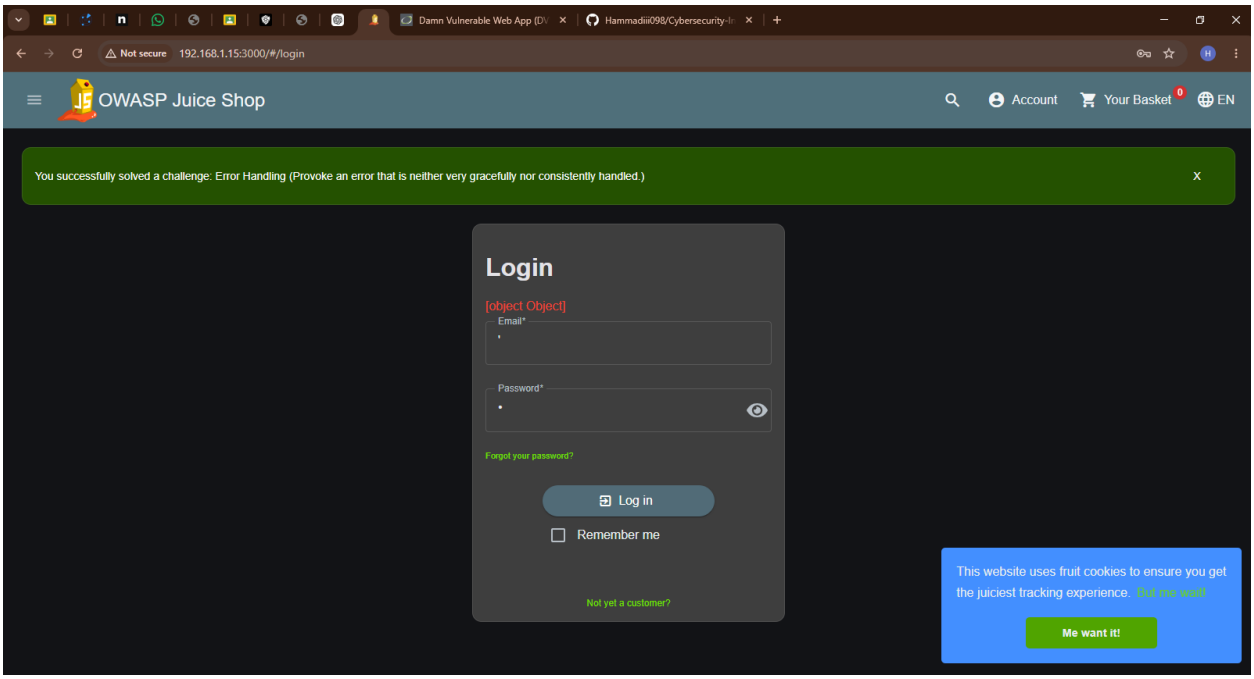
What Happens

The SQL query becomes:

```
SELECT * FROM Users
WHERE email = '' OR 1=1;--'
AND password = '...'
AND deletedAt IS NULL
```

Explanation

- `' OR 1=1` always evaluates to TRUE
- `--` comments out the remaining query
- Authentication is bypassed



Alternative Payload

admin@juice-sh.op' --

Admin Panel

After login, open:

http://localhost:3000/#/administration

B. Cross-Site Scripting (XSS) — DOM XSS

Search Bar XSS

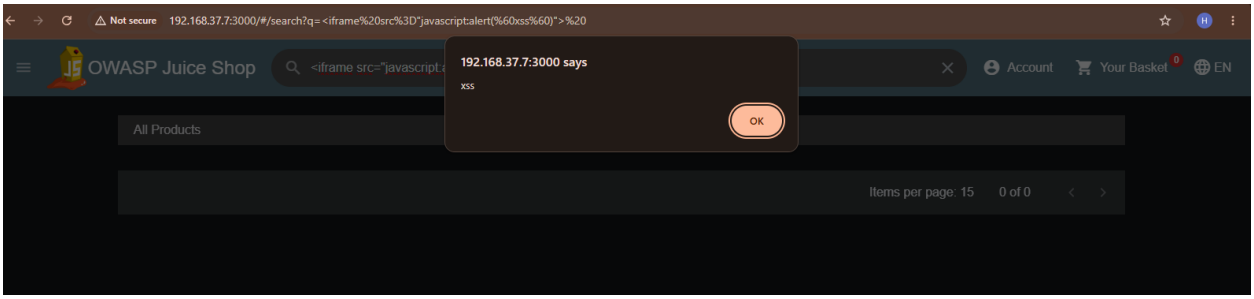
1. Click the Search icon
2. Enter:

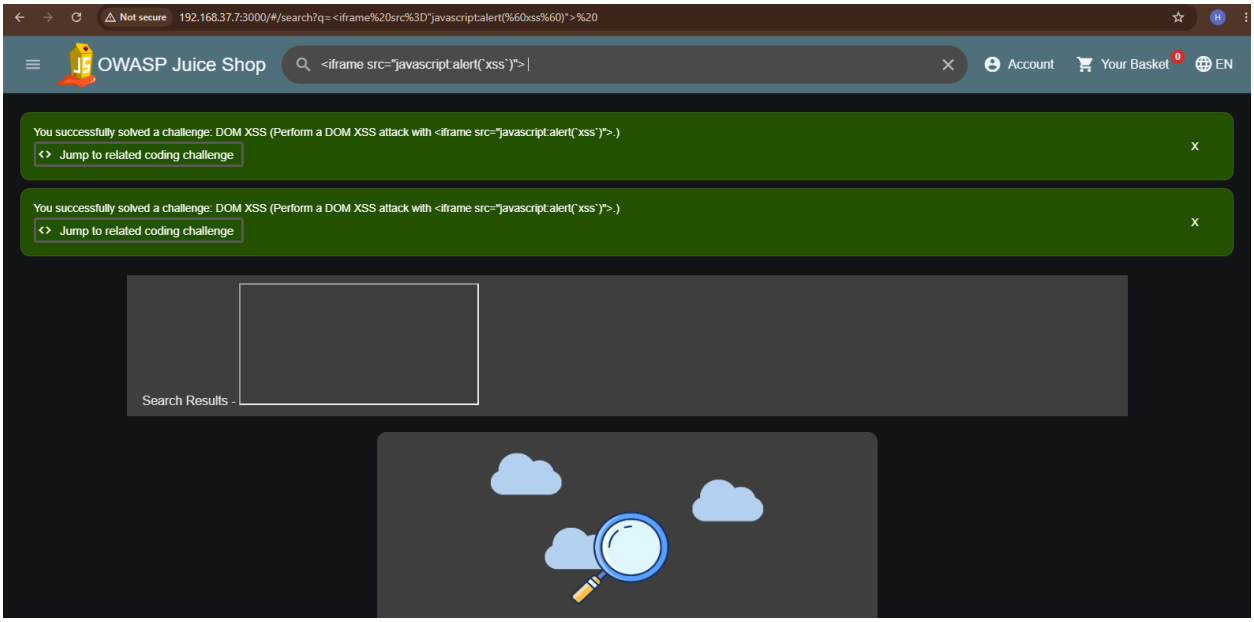
```
<iframe src="javascript:alert(`xss`)">
```

3. Press Enter

Result

JavaScript executes in the browser due to unsanitized DOM rendering.





Reflected XSS — Track Order Page

Inject the following into the tracking field:

```
<script>alert('XSS');</script>
```

C. Browser Developer Tools Inspection

Open DevTools

- Right-click → Inspect
- Or press `F12`

Areas to Inspect

Sources Tab

Search inside `juice-shop.min.js` for:

- admin
- password
- token
- API endpoints

Network Tab

Inspect:

- API responses
- JWT tokens
- User data

Application Tab

Check:

- Local Storage
- Session Storage

D. Automated Scanning with OWASP ZAP

Install ZAP

Download from:

https://www.zaproxy.org/download/

Quick Scan

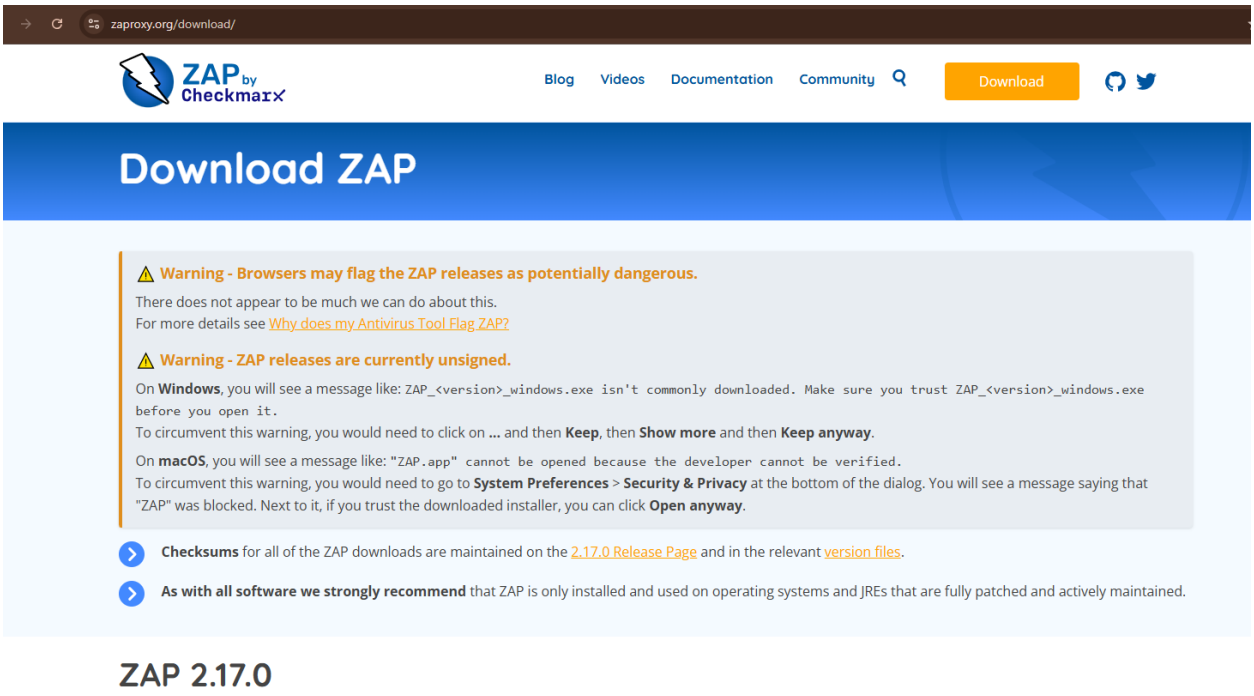
```
Tools → Quick URL Scan
```

Target:

http://localhost:3000

Full Scan

```
File → New Session
Sites Tree → Right-click → Attack → Active Scan
```



Review Findings

Check the **Alerts** tab:

- High
- Medium
- Low risk vulnerabilities

Export Report

`Report → Generate Report → HTML`

E. Weak Password Storage

Juice Shop intentionally uses weak MD5 hashing.

Example Endpoints

```
/api/Users
/rest/product/search
```

Crack MD5 Hashes

Use:

https://crackstation.net

Known Credentials

Email	Password
admin@juice-sh.op	admin123
jim@juice-sh.op	ncc-1701
bender@juice-sh.op	OhG0dPlease1nsertLol

Step 3: Document Findings

#	Vulnerability	Location	Severity	Description

1	SQL Injection	/rest/user/login	Critical	Authentication bypass using SQL payload
2	DOM XSS	Search Bar	High	Unsanitized rendering allows script execution
3	Reflected XSS	Track Order Page	High	User input reflected without sanitization
4	Weak Password Hashing	User Database	High	MD5 hashes are easily crackable
5	Exposed Admin Section	/administration	Medium	Weak access control
6	JWT Weak Secret	/rest/user/login	Medium	Predictable JWT secret
7	CORS Misconfiguration	API Endpoints	Medium	Overly permissive CORS
8	Missing Security Headers	All Pages	Low	Missing CSP and clickjacking protections

Areas of Improvement

- Parameterized queries
- Input validation
- Output encoding
- bcrypt/argon2 password hashing
- Strong JWT secrets
- RBAC implementation
- Helmet.js security headers
